

# Geometric Background

Fundamentals of Robotics  
University of Trento

TEACHER: Luigi Palopoli



## Matlab Implementation of rotations

Still on Rotation Matrices

Composition

Parametrisation

Quaternions

Homogeneous Transformations

Matlab Examples for Homogeneous Transformations

# Fundamental Rotation Matrices

We've seen how to define a rotation matrix about the z-axis.

$$R_z(\alpha) = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Matlab translation: (????)

# Fundamental Rotation Matrices

We've seen how to define a rotation matrix about the z-axis.

$$R_z(\alpha) = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Matlab translation: (????)

## zRot

```
1 % Function that returns the Rotation about z axis
2 function [Rz] = zRot(alpha)
3     Rz = [
4         cos(alpha), -sin(alpha), 0;
5         sin(alpha), cos(alpha), 0;
6         0, 0, 1
7     ];
8 end
```

We can do the same for the other fundamental rotation matrices.



# Fundamental Rotation Matrices

Rotation about the y-axis

# Fundamental Rotation Matrices

Rotation about the y-axis

yRot

```
1 % Function that returns the Rotation about z axis
2 function [Ry] = yRot(beta)
3     Ry = [
4         cos(beta), 0, sin(beta);
5         0, 1, 0;
6         -sin(beta), 0, cos(beta)
7     ];
8 end
```

Rotation about the x-axis

# Fundamental Rotation Matrices

## Rotation about the y-axis

### yRot

```
1 % Function that returns the Rotation about z axis
2 function [Ry] = yRot(beta)
3     Ry = [
4         cos(beta), 0, sin(beta);
5         0, 1, 0;
6         -sin(beta), 0, cos(beta)
7     ];
8 end
```

## Rotation about the x-axis

### xRot

```
1 % Function that returns the Rotation about z axis
2 function [Rx] = xRot(gamma)
3     Rx = [
4         1, 0, 0;
5         0, cos(gamma), -sin(gamma);
6         0, sin(gamma), cos(gamma)
7     ];
8 end
```

# Draw a Reference Frame

Let's now draw and rotate a frame.

The drawing can be created using the following script:

## scriptDraw

```

1  close all;                                %% CLOSES ALL WINDOWS
2  view(3);                                  %% Uses the default line of sight
    for 2D or 3D plots
3  daspect([1 1 1]);                        %% Sets the three scales of the axes
4  plot3([0,1],[0,0],[0,0], 'Color',[1,0,0]); %% Plots the first axis (color red)
5  hold on                                   %% Holds the image in the plot
6  plot3([0,0],[0,1],[0,0], 'Color',[0,1,0]); %% Plots the y axis
7  plot3([0,0],[0,0],[0,1], 'Color',[0,0,1]); %% Plots the z axis
8
9  v1 = zRot(pi/4)*[1;0;0]                  %% First vector oriented along the x
    axis rotated by 45 deg around z
10 v2 = zRot(pi/4)*[0;1;0]                  %% Second vector oriented along the
    y axis rotated by 45deg around z
11 v3 = zRot(pi/4)*[0;0;1]                  %% Third vector in oriented along
    the z axis rotated by 45deg around z
12
13 %% Plot the three rotated vectors
14 plot3([0,v1(1)],[0,v1(2)],[0,v1(3)], 'Color',[1,0,0], 'LineStyle','—');
15 plot3([0,v2(1)],[0,v2(2)],[0,v2(3)], 'Color',[0,1,0], 'LineStyle','—');
16 plot3([0,v3(1)],[0,v3(2)],[0,v3(3)], 'Color',[0,0,1], 'LineStyle','—');
    
```

# Outline

Matlab Implementation of rotations

## Still on Rotation Matrices

Composition

Parametrisation

Quaternions

Homogeneous Transformations

Matlab Examples for Homogeneous Transformations

# Outline

Matlab Implementation of rotations

Still on Rotation Matrices

Composition

Parametrisation

Quaternions

Homogeneous Transformations

Matlab Examples for Homogeneous Transformations

# Composition of Rotation Matrices

Let us start with an example.

- ▶ Consider three different frames sharing the same origin:  $O - x_0y_0z_0$ ,  $O - x_1y_1z_1$ ,  $O - x_2y_2z_2$
- ▶ Consider a point  $P$  with coordinates  $\mathbf{p}^0$ ,  $\mathbf{p}^1$ , and  $\mathbf{p}^2$  in the three frames.
- ▶ Let  $R_i^j$  denote the rotation matrix of Frame  $i$  with respect to Frame  $j$ .
- ▶ We have:

$$\mathbf{p}^1 = R_2^1 \mathbf{p}^2$$

$$\mathbf{p}^0 = R_1^0 \mathbf{p}^1$$

$$\mathbf{p}^0 = R_2^0 \mathbf{p}^2$$

- ▶ By simple substitution and comparison,

$$\mathbf{p}^0 = R_1^0 R_2^1 \mathbf{p}^2$$

$$\mathbf{p}^1 = R_2^1 \mathbf{p}^2 \rightarrow$$

$$R_2^0 = R_1^0 R_2^1$$

# Composition of Rotation Matrices

We can interpret the relation  $R_2^0 = R_1^0 R_2^1$  as being obtained in the two following steps:

1. First, rotate  $O - x_0y_0z_0$  so as to align it with frame  $O - x_1y_1z_1$  (matrix  $R_1^0$ ).
2. Then, rotate again so as to align frame  $O - x_1y_1z_1$  with  $O - x_2y_2z_2$  (matrix  $R_2^1$ ).

Generally speaking:

## Composition of Rotations

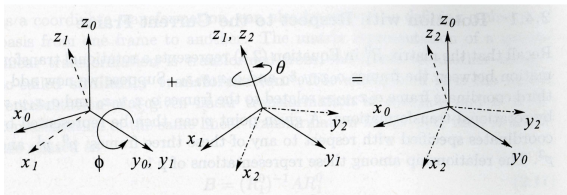
A rotation can be expressed as a sequence of partial rotations, each one defined with respect to the previous one. The frame from which the rotation occurs is termed the *current frame*. The sequence is obtained by post-multiplying the matrices associated with each rotation.

In view of the properties of rotation matrices, we have:

$$R_i^j = (R_j^i)^{-1} = (R_j^i)^T$$

# Composition of Rotation Matrices - Example

An example of a sequence of rotations about axes in *the current frame* is below.



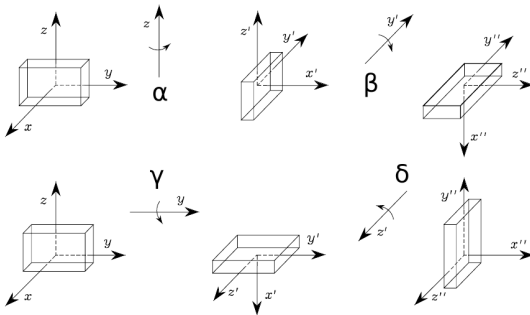
- ▶ We first rotate by  $\phi$  about the current  $y$ -axis.
- ▶ Then we rotate by  $\theta$  about the current  $z$ -axis.

Result:

$$R_2^0 = R_y(\phi)R_z(\theta).$$

# Composition of Rotation Matrices - Another example

An example of a sequence of rotations about axes in *the current frame* is below.



The resulting rotation matrix is:

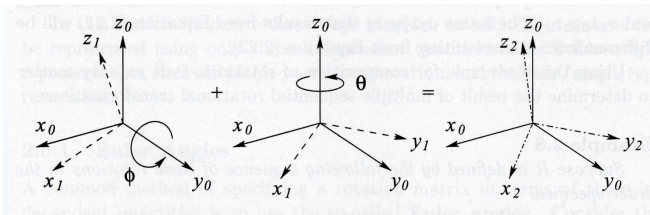
$$R_4^0 = R_z(\alpha)R_y(\beta)R_y(\gamma)R_z(\delta).$$

The order is absolutely important!

$$R_z(\alpha)R_y(\beta)R_y(\gamma)R_z(\delta) \neq R_y(\gamma)R_z(\delta)R_z(\alpha)R_y(\beta).$$

# Rotation about fixed axes

- What if we want to specify the sequence of rotations about fixed axes
- An example of a sequence of rotations *in a fixed frame* is below



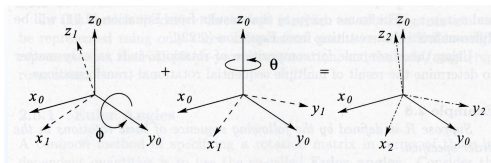
# Rotation about fixed axes

- ▶ To see how rotations can be composed about a fixed axis, let's use a simple fact:
- ▶ Consider two rotated frames  $O - x_0y_0z_0$  and  $O - x_1y_1z_1$ .
- ▶ For a generic point  $P$  we have:  $p^0 = R_1^0 p^1$
- ▶ Consider a generic linear application  $A$  defined in the  $O - x_0y_0z_0$  frame.
- ▶ For a generic point  $P$ , its transformed point we have:

$$q^0 = Ap^0$$

$$q^1 = R_0^1 Ap^0 = R_0^1 AR_1^0 p^1 = (R_1^0)^{-1} AR_1^0 p^1$$

# Rotation about fixed axes - back to the example



We have two rotations:

- ▶ The first rotation is  $R_1^0 = R_y(\phi)$  around the  $y_0$ -axis.
- ▶ The rotation  $\bar{R}_2^1$ , which happens around the  $z_0$ -axis (not the  $z_1$ -axis as in the case of the current frame).

$$R_2^0 = R_1^0 \bar{R}_2^1.$$

We know how the latter is done in the current axes, so we can use the result above:

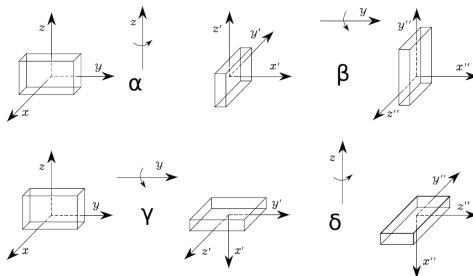
$$\bar{R}_2^1 = (R_1^0)^{-1} R_z(\theta) R_1^0,$$

which leads to

$$R_2^0 = R_1^0 \bar{R}_2^1 = R_1^0 (R_1^0)^{-1} R_z(\theta) R_1^0 = R_z(\theta) R_y(\phi)$$

# Rotation about fixed axes - back to the example

We can conclude that when the composition of rotations happens with fixed axes, we have to pre-multiply instead of post-multiplying (as is the case for rotations in the current frame). Consider the example:



We have:

$$R_4^0 = R_z(\delta)R_y(\gamma)R_y(\beta)R_z(\alpha)$$

# Rotation about current axes - Example

Consider the rotation composed as follows:

1. A rotation of  $\theta$  about the current x-axis.
2. A rotation of  $\phi$  around the current z-axis.

The result is:

$$R_2^0 = R_x(\theta)R_z(\phi).$$

A point expressed as  $p^2$  in the frame  $O - x_2y_2z_2$  corresponds to  $p^0 = R_2^0 p^2$  in the initial frame.

## A More Complex Example

Consider the rotation composed as follows:

1. A rotation of  $\theta$  about the current x-axis.
2. A rotation of  $\phi$  around the current z-axis.
3. A rotation of  $\alpha$  about the *fixed*  $z_0$ -axis.

Following the same procedure as above, we have:

The result is:

$$R_3^0 = R_1^0 R_2^1 \bar{R}_3^2 = R_x(\theta) R_z(\phi) \bar{R}_3^2.$$

where  $\bar{R}_3^2$  denotes the notation with respect to the initial fixed axes. The effect of the rotation about the fixed frame in the current frame

$O - x_2 y_2 z_2$  is given by:

$$\bar{R}_3^2 = (R_1^0 R_2^1)^{-1} R_z(\alpha) R_1^0 R_2^1 = (R_x(\theta) R_z(\phi))^{-1} R_z(\alpha) R_x(\theta) R_z(\phi)$$

which leads us to

$$R_3^0 = R_z(\alpha) R_x(\theta) R_z(\phi).$$

# The Final Example

Consider the rotation composed as follows:

1. A rotation of  $\theta$  about the current x-axis.
2. A rotation of  $\phi$  around the current z-axis.
3. A rotation of  $\alpha$  about the *fixed*  $z_0$ -axis.
4. A rotation of  $\beta$  around the current y-axis.
5. A rotation of  $\delta$  about the fixed  $x_0$ -axis.

Following the same procedure that we have seen before, we get:

$$R_r^0 = R_x(\delta)R_z(\alpha)R_x(\theta)R_z(\phi)R_y(\beta).$$

Please verify this yourself with paper and a pencil.

# Outline

Matlab Implementation of rotations

Still on Rotation Matrices

Composition

Parametrisation

Quaternions

Homogeneous Transformations

Matlab Examples for Homogeneous Transformations

# Parametrisation of Rotation Matrices

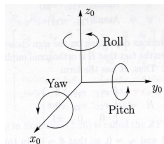
- ▶ A 3D rotation matrix has 9 elements, but in fact a rotation is completely specified by three parameters.
- ▶ Generally speaking, a rotation is an element of  $SO(m)$  that is completely specified by  $m(m-1)/2$  parameters.
  - ▶ 1 parameter for  $SO(2)$
  - ▶ 3 parameters for  $SO(3)$
- ▶ A 3D rotation is completely specified by three rotations about different axes, such that no two adjacent rotations are about the same axis.
- ▶ As a result, there are 12 parametrisations with three rotations:
 

|     |                                                            |
|-----|------------------------------------------------------------|
| 3 × | Any of the three axes is acceptable for the first rotation |
| 2 × | Rule out the axis used for the first rotation              |
| 2 = | Rule out the axis used for the second rotation             |
|     | 12                                                         |

- ▶ We will mention the most common ones.

# Euler Angles

- ▶ Euler angles are the angles related to three rotations about the current axis.
- ▶ A very popular convention is the use of  $XYZ$  rotations about fixed axes, which is known in the aeronautical world as roll-pitch-yaw because these rotations denote the change in attitude of an aircraft with respect to three fixed axes.



- ▶ Since the rotation is about fixed axes, we have to pre-multiply:

$$R = R_z(\phi)R_y(\theta)R_x(\psi)$$

- ▶  $\psi$  is the roll angle,  $\theta$  is the pitch angle,  $\phi$  is the yaw angle.

# Euler - Matlab

We can create any rotation matrix by specifying three angles. One way is the so-called triple of Euler angles (three rotations about the current axes: ZYZ).

$$R(\phi, \theta, \psi) = R_z(\phi)R_{y'}(\theta)R_{z''}(\psi)$$

The corresponding Matlab listing is:

## eulerRot

```
1 function [R] = eulerRot(phi, theta, psi)
2
3 R = zRot(phi)*yRot(theta)*zRot(psi);
4
5 end
```

The function can be tested through the following script:

## scriptDrawEuler

```
1  close all;
2
3  view(3);
4  daspect([1 1 1]);
5  plot3([0,1],[0,0],[0,0], 'Color',[1,0,0]);
6  hold on
7  plot3([0,0],[0,1],[0,0], 'Color',[0,1,0]);
8  plot3([0,0],[0,0],[0,1], 'Color',[0,0,1]);
9
10 R = eulerRot(pi/6, pi/6, 0);
11
12 v1 = R*[1;0;0]
13 v2 = R*[0;1;0]
14 v3 = R*[0;0;1]
15
16 plot3([0,v1(1)],[0,v1(2)],[0,v1(3)], 'Color',[1,0,0], 'LineStyle','—');
17 plot3([0,v2(1)],[0,v2(2)],[0,v2(3)], 'Color',[0,1,0], 'LineStyle','—');
18 plot3([0,v3(1)],[0,v3(2)],[0,v3(3)], 'Color',[0,0,1], 'LineStyle','—');
```

# RPY Angles

- ▶ Introduce the notation  $c_\alpha = \cos(\alpha)$ ,  $s_\alpha = \sin(\alpha)$ .
- ▶ It is easy to see that:

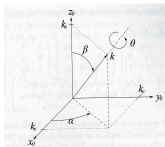
$$R = \begin{bmatrix} c_\phi c_\theta & c_\phi s_\theta s_\psi - s_\phi c_\psi & c_\phi s_\theta c_\psi + s_\phi s_\psi \\ s_\phi c_\theta & s_\phi s_\theta s_\psi + c_\phi c_\psi & s_\phi s_\theta c_\psi - c_\phi s_\psi \\ -s_\theta & c_\theta s_\psi & c_\theta c_\psi \end{bmatrix}$$

- ▶ Given a Rotation matrix  $R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$ , we can solve the inverse problem (try this as an exercise) and find two solutions:

$$\begin{aligned} \theta \in (-\pi/2, \pi/2) &\rightarrow \begin{cases} \phi = \text{atan2}(r_{21}, r_{11}) \\ \theta = \text{atan2}\left(-r_{31}, \sqrt{r_{32}^2 + r_{33}^2}\right) \\ \psi = \text{atan2}(r_{32}, r_{33}) \end{cases} \\ \theta \in (\pi/2, 3\pi/2) &\rightarrow \begin{cases} \phi = \text{atan2}(-r_{21}, -r_{11}) \\ \theta = \text{atan2}\left(-r_{31}, -\sqrt{r_{32}^2 + r_{33}^2}\right) \\ \psi = \text{atan2}(-r_{32}, -r_{33}) \end{cases} \end{aligned}$$

# Axis and rotation

- ▶ We can adopt a redundant parametrisation using an axis ( $\mathbf{k} = [k_x, k_y, k_z]$  with  $|\mathbf{k}| = 1$ ) and an angle ( $\theta$ ), resulting in four parameters overall.



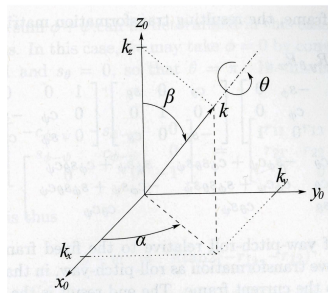
- ▶ The rotation  $R_{\mathbf{k}}(\theta)$  can be found by transforming the coordinates.
- ▶ We can find a new frame  $O - x_1y_1z_1$ , with  $z_1$  coincident with  $\mathbf{k}$ . We rotate  $O - x_0y_0z_0$  by  $\alpha$  about the  $z$ -axis and by  $\beta$  around the  $y$ -axis.
- ▶ Clearly  $R_1^0 = R_z(\alpha)R_y(\beta)$ . If a transformation  $A$  (e.g., a rotation) is expressed in the frame  $O - x_1y_1z_1$ , as in  $q^1 = Ap^1$ , we know that:

$$q^0 = (R_0^1)^{-1}AR_0^1p^0 = R_z(\alpha)R_y(\beta)A(R_z(\alpha)R_y(\beta))^{-1}$$

which leads to:

$$R_{\mathbf{k}}(\theta) = R_z(\alpha)R_y(\beta)R_z(\theta)R_y(-\beta)R_z(-\alpha)$$

# Axis and rotation



Using some trigonometry, we can find the  $\alpha$  and  $\beta$  angles.

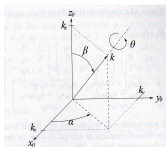
$$\sin(\alpha) = \frac{k_y}{\sqrt{k_x^2 + k_y^2}} \quad \cos(\alpha) = \frac{k_x}{\sqrt{k_x^2 + k_y^2}}$$

$$\sin(\beta) = \frac{\sqrt{k_x^2 + k_y^2}}{\sqrt{k_x^2 + k_y^2 + k_z^2}} \quad \cos(\beta) = \frac{k_z}{\sqrt{k_x^2 + k_y^2 + k_z^2}}$$

and

$$R_{\mathbf{k}}(\theta) = \begin{bmatrix} k_x^2(1 - c_\theta) + c_\theta & k_x k_y(1 - c_\theta) - k_z s_\theta & k_x k_z(1 - c_\theta) + k_y s_\theta \\ k_x k_y(1 - c_\theta) + k_z s_\theta & k_y^2(1 - c_\theta) + c_\theta & k_y k_z(1 - c_\theta) - k_x s_\theta \\ k_x k_z(1 - c_\theta) - k_y s_\theta & k_y k_z(1 - c_\theta) + k_x s_\theta & k_z^2(1 - c_\theta) + c_\theta \end{bmatrix}$$

# Axis and rotation



- We have specified a rotation matrix using four parameters ( $\mathbf{k}$  and  $\theta$ ), but there is also a constraint ( $k_x^2 + k_y^2 + k_z^2 = 1$ ), so the free parameters are actually three.
- From

$$R_{\mathbf{k}}(\theta) = \begin{bmatrix} k_x^2(1 - c_\theta) + c_\theta & k_x k_y(1 - c_\theta) - k_z s_\theta & k_x k_z(1 - c_\theta) + k_y s_\theta \\ k_x k_y(1 - c_\theta) + k_z s_\theta & k_y^2(1 - c_\theta) + c_\theta & k_y k_z(1 - c_\theta) - k_x s_\theta \\ k_x k_z(1 - c_\theta) - k_y s_\theta & k_y k_z(1 - c_\theta) + k_x s_\theta & k_z^2(1 - c_\theta) + c_\theta \end{bmatrix}$$

We can see that  $R_{-\mathbf{k}}(-\theta) = R_{\mathbf{k}}(\theta)$ .

- If we invert the axis and the angle, we get the same rotation.

**Sadly....**

With an axis and rotation, the rotation matrix is not unique and the representation is not invertible.

# Outline

Matlab Implementation of rotations

Still on Rotation Matrices

Composition

Parametrisation

Quaternions

Homogeneous Transformations

Matlab Examples for Homogeneous Transformations



# Quaternions

- Consider a real number  $x \in \mathbb{R}$ . What is its inverse?

$$\frac{1}{x} \cdot x = \rightarrow x^{-1} = \frac{1}{x}.$$

# Quaternions

- Consider a real number  $x \in \mathbb{R}$ . What is its inverse?

$$\frac{1}{x} \cdot x = \rightarrow x^{-1} = \frac{1}{x}.$$

- Consider an element of  $\mathbb{R}^2$ ,  $x = \begin{bmatrix} a \\ b \end{bmatrix}$ . Can we find its inverse? Yes, if we resort to complex numbers. Introduce  $z = a + ib$ , where  $i$  is the imaginary unit. What can we do? Introduce the complex conjugate  $z^* = a - ib$  and consider the number  $z_1 = \frac{z^*}{|z|^2}$ . We have:  $z \cdot z_1 = \frac{z \cdot z^*}{|z|^2} = 1$ . Hence:  $z^{-1} = z_1$ .

# Quaternions

- ▶ Consider a real number  $x \in \mathbb{R}$ . What is its inverse?

$$\frac{1}{x} \cdot x = \rightarrow x^{-1} = \frac{1}{x}.$$

- ▶ Consider an element of  $\mathbb{R}^2$ ,  $x = \begin{bmatrix} a \\ b \end{bmatrix}$ . Can we find its inverse? Yes, if we resort to complex numbers. Introduce  $z = a + ib$ , where  $i$  is the imaginary unit. What can we do? Introduce the complex conjugate  $z^* = a - ib$  and consider the number  $z_1 = \frac{z^*}{|z|^2}$ . We have:  $z \cdot z_1 = \frac{z \cdot z^*}{|z|^2} = 1$ . Hence:  $z^{-1} = z_1$ .
- ▶ We cannot repeat the same trick with  $\mathbb{R}^3 \dots$  but

# Quaternions

- ▶ Consider a real number  $x \in \mathbb{R}$ . What is its inverse?

$$\frac{1}{x} \cdot x = \rightarrow x^{-1} = \frac{1}{x}.$$

- ▶ Consider an element of  $\mathbb{R}^2$ ,  $x = \begin{bmatrix} a \\ b \end{bmatrix}$ . Can we find its inverse? Yes, if we resort to complex numbers. Introduce  $z = a + ib$ , where  $i$  is the imaginary unit. What can we do? Introduce the complex conjugate  $z^* = a - ib$  and consider the number  $z_1 = \frac{z^*}{|z|^2}$ . We have:  $z \cdot z_1 = \frac{z \cdot z^*}{|z|^2} = 1$ . Hence:  $z^{-1} = z_1$ .
- ▶ We cannot repeat the same trick with  $\mathbb{R}^3 \dots$  but
- ▶ We can with  $\mathbb{R}^4$ , using the so-called quaternions.

# Definition

## Definition of complex number

- ▶ Let 1 and  $i$  be the basis elements.
- ▶ Let the following axiom hold:  $i^2 = -1$ .
- ▶ *Complex numbers* are defined as a vector space over the reals, whose elements have the form  $z = x + iy$ , where  $x \in \mathbb{R}$ ,  $y \in \mathbb{R}$ .

## Definition of complex number

- ▶ Let 1 and  $i$  be the basis elements.
- ▶ Let the following axiom hold:  $i^2 = -1$ .
- ▶ *Complex numbers* are defined as a vector space over the reals, whose elements have the form  $z = x + iy$ , where  $x \in \mathbb{R}$ ,  $y \in \mathbb{R}$ .

## Definition of quaternion

- ▶ Let 1 and  $i, j, k$  be the basis elements.
- ▶ Let the following axiom hold:  $i^2 = j^2 = k^2 = ijk = -1$ .
- ▶ *The quaternions* are defined as a vector space over the reals, whose elements have the form  $q = q_0 + q_1i + q_2j + q_3k$ .
- ▶ A shorthand notation distinguishes the *scalar part*  $q_0$  and the *vector part*  $\mathbf{q} = q_1i + q_2j + q_3k$ :  $\mathbf{v} = q_0 + \mathbf{q}$ .
- ▶ An equivalent notation is  $[q_0, \mathbf{q}]$ .

# Background on vector products

## Scalar product (dot product)

Given two vectors  $\mathbf{u} = u_1\mathbf{i} + u_2\mathbf{j} + u_3\mathbf{k}$  and  $\mathbf{v} = v_1\mathbf{i} + v_2\mathbf{j} + v_3\mathbf{k}$ , the scalar product  $\mathbf{u} \cdot \mathbf{v}$  is the real number  $\mathbf{u} \cdot \mathbf{v} = u_1v_1 + u_2v_2 + u_3v_3$ . If  $\theta$  is the angle between the vectors, we have:  $\mathbf{u} \cdot \mathbf{v} = |\mathbf{u}| |\mathbf{v}| \cos \theta$ . The operation represents the projection of one vector onto the other and is commutative.

# Background on vector products

## Scalar product (dot product)

Given two vectors  $\mathbf{u} = u_1\mathbf{i} + u_2\mathbf{j} + u_3\mathbf{k}$  and  $\mathbf{v} = v_1\mathbf{i} + v_2\mathbf{j} + v_3\mathbf{k}$ , the scalar product  $\mathbf{u} \cdot \mathbf{v}$  is the real number  $\mathbf{u} \cdot \mathbf{v} = u_1v_1 + u_2v_2 + u_3v_3$ . If  $\theta$  is the angle between the vectors, we have:  $\mathbf{u} \cdot \mathbf{v} = |\mathbf{u}| |\mathbf{v}| \cos \theta$ . The operation represents the projection of one vector onto the other and is commutative.

## Vector product (cross product)

Given two vectors  $\mathbf{u} = u_1\mathbf{i} + u_2\mathbf{j} + u_3\mathbf{k}$  and  $\mathbf{v} = v_1\mathbf{i} + v_2\mathbf{j} + v_3\mathbf{k}$ , the cross product  $\mathbf{s} = \mathbf{u} \times \mathbf{v}$  is a vector orthogonal to both  $\mathbf{u}$  and  $\mathbf{v}$ , whose norm is given by  $|\mathbf{s}| = |\mathbf{u}| |\mathbf{v}| \sin \theta$ . We can show that

$$\mathbf{s} = \mathbf{u} \times \mathbf{v} = s_1\mathbf{i} + s_2\mathbf{j} + s_3\mathbf{k} \rightarrow$$

$$s_1 = u_2v_3 - u_3v_2$$

$$s_2 = u_3v_1 - u_1v_3$$

$$s_3 = u_1v_2 - u_2v_1$$

The vector product is anti-commutative:  $\mathbf{u} \times \mathbf{v} = -\mathbf{v} \times \mathbf{u}$ .

# Basic facts on multiplication

By applying the axioms  $i^2 = j^2 = k^2 = ijk = -1$  it is straightforward to see:

$$ij = k, \quad ji = -k$$

$$jk = i, \quad kj = -i$$

$$ki = j, \quad ik = -j$$

The rules are the same as for the cross product.

# Quaternion Product

We can take advantage of the vector notation and compute the product of two quaternions:

$$p = p_0 + \mathbf{p}$$

$$q = q_0 + \mathbf{q}$$

$$pq = p_0q_0 + p_0\mathbf{q} + q_0\mathbf{p} + \mathbf{p}\mathbf{q}$$

It is easy to see that:

$$pq = p_0q_0 - \mathbf{p} \cdot \mathbf{q} + p_0\mathbf{q} + q_0\mathbf{p} + \mathbf{p} \times \mathbf{q}$$

In other words, the product quaternion has scalar part  $p_0q_0 - \mathbf{p} \cdot \mathbf{q}$  and vector part  $p_0\mathbf{q} + q_0\mathbf{p} + \mathbf{p} \times \mathbf{q}$ . The quaternion product is associative under addition but *is not* commutative.

# Quaternion Conjugate

## Definition of quaternion conjugate

$$q = q_0 + q_1i + q_2j + q_3k \rightarrow q^* = q_0 - q_1i - q_2j - q_3k$$

By applying this definition and carrying out the computations, one obtains:

$$\begin{aligned} qq^* &= (q_0 + \mathbf{q})(q_0 - \mathbf{q}) = \\ &= q_0^2 + q_0\mathbf{q} - q_0\mathbf{q} - q = \\ &= q_0^2 + q_1^2 + q_2^2 + q_3^2 \end{aligned}$$

## Definition of quaternion length

The quantity  $|q| = \sqrt{qq^*} = \sqrt{q_0^2 + q_1^2 + q_2^2 + q_3^2}$  is defined as the quaternion length.

# Quaternion Inverse

Based on the definition above, a quaternion (except 0) has a *unique* inverse  $q^{-1} = \frac{q^*}{|q|^2}$ :

$$q \frac{q^*}{|q|^2} = \frac{qq^*}{|q|^2} = 1.$$

For simplicity, we shall henceforth consider only unit-length quaternions.

# Unit-length Quaternions

Consider a unit-length quaternion:  $q = [\eta, \epsilon]$  with  $\eta$  being the scalar part

and  $\epsilon = \begin{bmatrix} \epsilon_x \\ \epsilon_y \\ \epsilon_z \end{bmatrix}$  being the vector part.

It is easy to see that we can express the quaternion as  $[\cos(\theta/2), \sin(\theta/2) \mathbf{k}]$  where  $\mathbf{k}$  is a unit vector:

$$\mathbf{k} = \frac{\epsilon}{|\epsilon|}$$

$$\theta = 2 \operatorname{atan2}(|\epsilon|, \eta)$$

## Fact

Consider the unit-length quaternion  $q = [\cos(\theta/2), \sin(\theta/2) \mathbf{k}]$ . The quaternion represents a rotation of an angle  $\theta$  about the vector  $\mathbf{k}$ .

Why?

# Quaternions and Rotations

Let  $q = [\eta, \epsilon]$ . By computing  $p' = qpq^*$ , we obtain:

$$\begin{aligned} p' &= [\eta, \epsilon][0, \mathbf{p}][\eta, -\epsilon] = \\ &= [-\eta(\epsilon \cdot \mathbf{p}) + (\epsilon \times \mathbf{p}) \cdot \mathbf{p} + \eta(\epsilon \cdot \mathbf{p}), \\ &\quad -(\epsilon \times \mathbf{p}) \times \epsilon - \eta(\mathbf{p} \times \epsilon) + \eta(\epsilon \times \mathbf{p}) + \eta^2 \mathbf{p} + \epsilon(\epsilon \cdot \mathbf{p})] = \\ &\dots \text{ after some computation } \dots \\ &= [0, (\eta^2 - \epsilon \cdot \epsilon)\mathbf{p} + 2\eta(\epsilon \times \mathbf{p}) + 2(\epsilon \cdot \mathbf{p})\epsilon] \end{aligned}$$

# Quaternions and Rotations

Let  $q = [\eta, \epsilon]$ . By computing  $p' = qpq^*$ , we obtain:

$$\begin{aligned} p' &= [\eta, \epsilon][0, \mathbf{p}][\eta, -\epsilon] = \\ &= [-\eta(\epsilon \cdot \mathbf{p}) + (\epsilon \times \mathbf{p}) \cdot \mathbf{p} + \eta(\epsilon \cdot \mathbf{p}), \\ &\quad -(\epsilon \times \mathbf{p}) \times \epsilon - \eta(\mathbf{p} \times \epsilon) + \eta(\epsilon \times \mathbf{p}) + \eta^2 \mathbf{p} + \epsilon(\epsilon \cdot \mathbf{p})] = \\ &\quad \dots \text{ after some computation } \dots \\ &= [0, (\eta^2 - \epsilon \cdot \epsilon)\mathbf{p} + 2\eta(\epsilon \times \mathbf{p}) + 2(\epsilon \cdot \mathbf{p})\epsilon] \end{aligned}$$

By considering  $[\eta, \epsilon] = [\cos(\theta/2), \sin(\theta/2)\mathbf{k}]$  and applying some trigonometry, we obtain:

$$p' = [0, c_\theta \mathbf{p} + s_\theta \mathbf{k} \times \mathbf{p} + (1 - c_\theta)(\mathbf{k} \cdot \mathbf{p})\mathbf{k}]$$

where  $c_\theta = \cos(\theta)$  and  $s_\theta = \sin(\theta)$ .



# Quaternions and Rotations

The formula:

$$\mathbf{p}' = c_\theta \mathbf{p} + s_\theta \mathbf{k} \times \mathbf{p} + (1 - c_\theta)(\mathbf{k} \cdot \mathbf{p})\mathbf{k},$$

is called the Rodrigues formula and is *very important*. After some tedious computation, we find that the Rodrigues formula is equivalent to

$$\mathbf{p}' = R_{\mathbf{k}}(\theta)\mathbf{p}$$

where  $R_{\mathbf{k}}(\theta)$  is the matrix corresponding to a rotation of  $\theta$  about the axis  $\mathbf{k}$

$$R_{\mathbf{k}}(\theta) = \begin{bmatrix} k_x^2(1 - c_\theta) + c_\theta & k_x k_y(1 - c_\theta) - k_z s_\theta & k_x k_z(1 - c_\theta) + k_y s_\theta \\ k_x k_y(1 - c_\theta) + k_z s_\theta & k_y^2(1 - c_\theta) + c_\theta & k_y k_z(1 - c_\theta) - k_x s_\theta \\ k_x k_z(1 - c_\theta) - k_y s_\theta & k_y k_z(1 - c_\theta) + k_x s_\theta & k_z^2(1 - c_\theta) + c_\theta \end{bmatrix}$$

# Summary

Given a unit-length quaternion  $q = [\eta, \epsilon] = [\cos(\theta/2), \sin(\theta/2) \mathbf{k}]$ ,

$$\mathbf{k} = \frac{\epsilon}{|\epsilon|}$$

$$\alpha = 2 \arctan 2(|\epsilon|, \eta)$$

we can associate it with a rotation of  $\theta$  about the axis  $\mathbf{k}$ . The corresponding rotation matrix is given by:

$$R_{\mathbf{k}}(\theta) = \begin{bmatrix} k_x^2(1 - c_\theta) + c_\theta & k_x k_y(1 - c_\theta) - k_z s_\theta & k_x k_z(1 - c_\theta) + k_y s_\theta \\ k_x k_y(1 - c_\theta) + k_z s_\theta & k_y^2(1 - c_\theta) + c_\theta & k_y k_z(1 - c_\theta) - k_x s_\theta \\ k_x k_z(1 - c_\theta) - k_y s_\theta & k_y k_z(1 - c_\theta) + k_x s_\theta & k_z^2(1 - c_\theta) + c_\theta \end{bmatrix}$$

The parameters of the quaternion are three:  $\theta$ ,  $k_x$ ,  $k_y$ ,  $k_z$ , or, equivalently,  $\eta$  and  $\epsilon$ , with the constraint:

$$\eta^2 + \epsilon_x^2 + \epsilon_y^2 + \epsilon_z^2 = 1.$$

# From Rotation Matrix to Quaternions

- Given a rotation matrix  $R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$ , we can solve the inverse problem:

$$\eta = \frac{1}{2} \sqrt{r_{11} + r_{22} + r_{33} + 1}$$

$$\epsilon = \frac{1}{2} \begin{bmatrix} \text{sgn}(r_{32} - r_{23}) \sqrt{r_{11} - r_{22} - r_{33} + 1} \\ \text{sgn}(r_{13} - r_{31}) \sqrt{r_{22} - r_{33} - r_{11} + 1} \\ \text{sgn}(r_{21} - r_{12}) \sqrt{r_{33} - r_{11} - r_{22} + 1} \end{bmatrix} \quad \text{where}$$

$$\text{sgn}(x) = \begin{cases} 1 & x \geq 0 \\ -1 & x < 0 \end{cases}$$

## Advantages of Quaternions

- Quaternions have many numerical advantages.
- By using quaternions we resolve the issues of invertibility and non-uniqueness if we restrict ourselves to one of the two hemispheres.
- ... as well as many other benefits (e.g., linear interpolation on spheres).

# Composition of Rotations

Consider rotations associated with the unit-length quaternions  $q_0$  and  $q_1$ . The first rotates a generic point  $\mathbf{p}$  into  $\mathbf{p}'$ , and the second moves  $\mathbf{p}'$  into  $\mathbf{p}''$ . Introduce the quaternions  $p = [0, \mathbf{p}]$ ,  $p' = [0, \mathbf{p}']$ , and  $p'' = [0, \mathbf{p}'']$ . We have:

$$\begin{aligned} p' &= q_0 p q_0^* \\ p'' &= q_1 p' q_1^* = \\ &= q_1 q_0 p q_0^* q_1^* = \\ &= (q_1 q_0) p (q_1 q_0)^*. \end{aligned}$$

The last step holds because  $q_0^* q_1^* = (q_1 q_0)^*$ .

Hence, *the composition of two rotations corresponds to the product of the quaternions.*

# Quaternions

- We can easily introduce operations on quaternions that produce the same results as the most common operations on rotation matrices.

| Operation   | Rotation matrices                  | Quaternions                                                                                                                                                                                                                  |
|-------------|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Inversion   | $R^{-1} = R^T$                     | $q = (\eta, \epsilon) \rightarrow q^{-1} \leftrightarrow (\eta, -\epsilon)$                                                                                                                                                  |
| Composition | $R_1, R_2 \rightarrow R = R_1 R_2$ | $q_1 = (\eta_1, \epsilon_1), q_2 = (\eta_2, \epsilon_2) \rightarrow$<br>$q = q_1 \cdot q_2 \leftrightarrow (\eta_1 \eta_2 - \epsilon_1^T \epsilon_2, \eta_1 \epsilon_2 + \eta_2 \epsilon_1 + \epsilon_1 \times \epsilon_2).$ |

# Outline

Matlab Implementation of rotations

Still on Rotation Matrices

- Composition

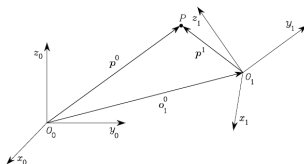
- Parametrisation

- Quaternions

**Homogeneous Transformations**

Matlab Examples for Homogeneous Transformations

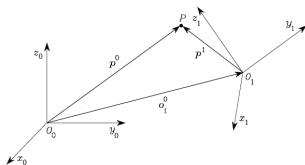
# Position of a Frame in 3D



- ▶ At the beginning of this lecture, we saw that a rigid body is identified by the position and orientation of a frame attached to it.
- ▶ Then, we ignored translation and considered only rotation (with the origins of the frames coincident).
- ▶ It is now time to return to the general case.
- ▶ It is straightforward to see that

$$\mathbf{p}^0 = \mathbf{o}_1^0 + R_1^0 \mathbf{p}^1.$$

# Direct and Inverted Coordinate Transformation



- The inverse relation of

$$\mathbf{p}^0 = \mathbf{o}_1^0 + R_1^0 \mathbf{p}^1$$

is

$$\begin{aligned} \mathbf{p}^1 &= -R_1^{0T} \mathbf{o}_1^0 + R_1^{0T} \mathbf{p}^0 \\ &= -R_0^1 \mathbf{o}_1^0 + R_0^1 \mathbf{p}^0 \end{aligned}$$

- This type of representation is rather cumbersome.

# Homogeneous Coordinates

- ▶ We need a compact way to represent both rotation and translation.
- ▶ The trick is simple: create the *homogeneous representation*  $\tilde{\mathbf{p}}$  of  $\mathbf{p}$  by appending a fictitious 1 to the vector.

$$\tilde{\mathbf{p}} = \begin{bmatrix} \mathbf{p} \\ 1 \end{bmatrix}$$

- ▶ We can then construct the following  $4 \times 4$  matrix:

$$A_1^0 = \begin{bmatrix} R_1^0 & \mathbf{o}_1^0 \\ \mathbf{0}^T & 1 \end{bmatrix}$$

which is called the *homogeneous transformation matrix*.

- ▶ The transformation from frame  $O_0 - x_0y_0z_0$  to  $O_1 - x_1y_1z_1$  can be written as

$$\tilde{\mathbf{p}}^0 = A_1^0 \tilde{\mathbf{p}}^1.$$

# Homogeneous Coordinates

- ▶ A first advantage is that we can express the coordinate transformation in matrix multiplication form:

$$\tilde{\mathbf{p}}^0 = A_1^0 \tilde{\mathbf{p}}^1.$$

- ▶ The inverse is simply given by

$$\begin{aligned}\tilde{\mathbf{p}}^1 &= A_0^1 \tilde{\mathbf{p}}^0 \\ &= (A_1^0)^{-1} \tilde{\mathbf{p}}^0\end{aligned}$$

- ▶ It is easy to see that

$$A_0^1 = (A_1^0)^{-1} = \begin{bmatrix} R_1^{0T} & -R_1^{0T} \mathbf{o}_1^0 \\ \mathbf{0}^T & 1 \end{bmatrix}$$

# Homogeneous Coordinates

- ▶ Unlike rotation matrices, homogeneous transformations are not orthogonal, therefore

$$A^{-1} \neq A^T.$$

- ▶ The homogeneous formalism allows us to generalise to roto-translations everything we have seen for rotations.
- ▶ For example, for a sequence of transformations in the current axes, we can write:

$$\tilde{\mathbf{p}}^0 = A_1^0 A_2^1 \dots A_n^{n-1} \tilde{\mathbf{p}}^n.$$

- ▶ The homogeneous transformations form a group called the special Euclidean group  $SE(3)$ :  $SE(3) = \mathbb{R}^3 \times SO(3)$ .

# Outline

Matlab Implementation of rotations

Still on Rotation Matrices

- Composition

- Parametrisation

- Quaternions

Homogeneous Transformations

- Matlab Examples for Homogeneous Transformations

# Homogeneous Transformation

Similarly, we can create homogeneous transformation matrices:

$$A_1^0 = \begin{bmatrix} R(\phi, \theta, \psi) & O_1^0 \\ O & 1 \end{bmatrix}$$

The corresponding Matlab listing is:

## homogeneousTrans

```
1 %homogeneous transformation
2 function [T] = homogeneousTrans(phi , theta , psi ,Ov)
3 R = eulerRot(phi , theta , psi );
4
5 T = [R, Ov;
6      zeros(1,3) , 1]
7 end
```

# Homogeneous Transformation – Test

The Matlab script to test the function is the following:

## scriptDrawHom

```

1  close all;
2
3  view(3);
4  daspect([1 1 1]);
5  plot3([0,1],[0,0],[0,0],'Color',[1,0,0]);
6  hold on
7  plot3([0,0],[0,1],[0,0],'Color',[0,1,0]);
8  plot3([0,0],[0,0],[0,1],'Color',[0,0,1]);
9
10 T = homogeneousTrans(0, 0, pi/4,[1;1;1]);
11 O1 = T*[0;0;0;1];
12
13 v1 = T*[1;0;0;1]
14 v2 = T*[0;1;0;1]
15 v3 = T*[0;0;1;1]
16
17 plot3([O1(1),v1(1)],[O1(2),v1(2)],[O1(3),v1(3)'],'Color',[1,0,0],'LineStyle','—');
18 plot3([O1(1),v2(1)],[O1(2),v2(2)],[O1(3),v2(3)'],'Color',[0,1,0],'LineStyle','—');
19 plot3([O1(1),v3(1)],[O1(2),v3(2)],[O1(3),v3(3)'],'Color',[0,0,1],'LineStyle','—');

```

# Homogeneous Transformation – Test

We can equivalently use built-in Matlab functions:

## scriptDrawHom1

```
1  close all;
2
3  view(3);
4  daspect([1 1 1]);
5  plot3([0,1],[0,0],[0,0], 'Color',[1,0,0]);
6  hold on
7  plot3([0,0],[0,1],[0,0], 'Color',[0,1,0]);
8  plot3([0,0],[0,0],[0,1], 'Color',[0,0,1]);
9
10 T = makehgtform('translate',[1,1,1], 'zrotate',pi/4)
11 O1 = T*[0;0;0;1];
12
13 v1 = T*[1;0;0;1]
14 v2 = T*[0;1;0;1]
15 v3 = T*[0;0;1;1]
16
17 plot3([O1(1),v1(1)],[O1(2),v1(2)],[O1(3),v1(3)], 'Color',[1,0,0], 'LineStyle','—');
18 plot3([O1(1),v2(1)],[O1(2),v2(2)],[O1(3),v2(3)], 'Color',[0,1,0], 'LineStyle','—');
19 plot3([O1(1),v3(1)],[O1(2),v3(2)],[O1(3),v3(3)], 'Color',[0,0,1], 'LineStyle','—');
```

# A Function to Draw Reference Frames

I have downloaded a community file to draw axis frames.

## scriptTestTriad

```
1  close all
2  %Create some axis
3  axs = axes;
4
5  %Set 3D view
6  view(3);
7
8  %Set aspect ratio
9  daspect([1 1 1]);
10
11 %Draw a first reference frame that coincides with the current axis
12 h = triad('Parent', axs);
13
14 %Now draw a second reference frame obtained with a translation and a
15 %rotation about x
16 h1=triad('Parent',h,'Matrix',makehgtform('translate',[1,1,1],'xrotate',pi/4),'
           linewidth',3,'linestyle','—')
```

# Matlab Transformation Objects

Matlab provides an object to create a transformation, link it to the current axis, and then define child objects associated with it that are transformed accordingly.

## script TransformTest

```
1 clear all;
2 close all;
3 %Define the axis
4 axs=axes('XLim',[-1.5 1.5], 'YLim',[-1.5 1.5], 'ZLim',[-1.5 1.5]);
5 view(3)
6 grid on;
7 % Create an empty transformation referred
8 % to the axis just created
9 t = hgtransform('Parent',axs);
10 %Create a cylinder
11 [x,y,z] = cylinder([1,1],100);
12
13 h = surface(x,y,z, 'FaceColor', 'red');
14 %Associate the cylinder with the
15 %transformation object
16 set(h, 'Parent',t);
17 %Wait for a key to be pressed
18 pause
19 %create a rotation matrix
20 Rz = makehgtform('xrotate',2*pi/3);
21 %apply to the transformation object and to its children
22 set(t, 'Matrix',Rz);
23 %Update the children objects
24 drawnow;
```

# Matlab Transformation: Example with Frames

Let us consider a transformation with frames.

## script TransformTriads

```
1 clear all;
2 close all;
3 %Define the axis
4 axs=axes('XLim',[-2.5 1.5], 'YLim',[-2.5 2.5], 'ZLim',[-2.5 2.5]);
5 view(3)
6 grid on;
7 % Create an empty transformation referred
8 % to the axis just created
9 t = hgtransform('Parent',axs);
10 h = triad('Parent', t);
11 %Now draw a second reference frame obtained with a translation and a
12 %rotation about x
13 h1=triad('Parent',h,'Matrix',makehgtform('translate',[1,1,1], 'xrotate',pi/4))
14 plot3([0,1],[0,1],[0,1], 'Color',[1,0,0], 'Parent',h, 'linewidth',3, 'linestyle',
15 '—');
16 %Wait for a key to be pressed
17 pause
18 for angle = 0:0.5:360,
19 %create a rotation matrix
20 Rz = makehgtform('zrotate',deg2rad(angle));
21 %apply to the transformation object and to its children
22 set(t,'Matrix',Rz);
23 %Update the children objects
24 drawnow;
25 end
```